

Theory of Computation - Complete Notes

Coal India Limited - Management Trainee (Systems) | CBT Preparation

Header watermark: Asabhya Maanav

Table of Contents

How To Use These Notes	5
1. Foundations (Preliminaries)	6
1.1 Alphabets, Symbols and Strings	6
1.2 The Empty String (Epsilon)	6
1.3 Languages and Operations	7
1.4 Kleene Star, Kleene Plus and Sigma-star	7
1.5 Chomsky Hierarchy	7
2. Finite Automata	9
2.1 Deterministic Finite Automaton (DFA)	9
2.2 Non-deterministic Finite Automaton (NFA)	10
2.3 Epsilon-NFA	10
2.4 Conversions	11
2.5 DFA Minimization	11
2.6 Moore and Mealy Machines	12
3. Regular Expressions	13
3.1 Definition; operators and precedence	13
3.2 Algebraic identities	13
3.3 RE to Finite Automaton	13
3.4 Finite Automaton to RE	13
4. Properties of Regular Languages	15
4.1 Closure Properties	15
4.2 Pumping Lemma for Regular Languages	15
4.3 Myhill-Nerode Theorem	15
4.4 Decision Properties	16
4.5 Regular Grammars	16
5. Context-Free Grammars (CFG)	17
5.1 Formal definition	17
5.2 Derivations: leftmost and rightmost	17
5.3 Parse trees and yield	17
5.4 Ambiguity	17
5.5 CFG design for given languages	17
5.6 Simplification of CFGs	17

5.7 Normal Forms	18
5.8 Left recursion and left factoring	18
6. Pushdown Automata (PDA)	19
6.1 Formal definition (7-tuple); stack operations	19
6.2 Acceptance: final state vs empty stack	19
6.3 NPDA construction from CFG	19
6.4 Deterministic PDA and DCFL	19
6.5 Equivalence of CFG and PDA	19
6.6 NPDA more powerful than DPDA	19
7. Properties of Context-Free Languages	21
7.1 Closure properties	21
7.2 CFL intersect Regular = CFL	21
7.3 Pumping Lemma for CFLs	21
7.4 Decision properties of CFLs	21
8. Turing Machines (TM)	23
8.1 Formal definition (7-tuple)	23
8.2 TM as language acceptor; configurations	23
8.3 TM as computing device	23
8.4 Construction of simple TMs	23
8.5 Variants and equivalence	23
8.6 Universal Turing Machine	23
8.7 Church-Turing Thesis	24
9. Recursive and Recursively Enumerable Languages	25
9.1 Recursive (decidable) languages	25
9.2 Recursively enumerable (semi-decidable) languages	25
9.3 Closure properties	25
9.4 Recursive iff L and complement both RE	25
10. Decidability and Undecidability	26
10.1 Decidable problems for FA and CFG	26
10.2 Halting problem	26
10.3 Reductions (mapping reducibility)	26
10.4 Rice's Theorem	26
10.5 Post Correspondence Problem (PCP)	26
10.6 RE-but-not-recursive examples	26

11. Introduction to Complexity (Awareness)	28
11.1 Time and space complexity of TMs	28
11.2 Classes P and NP	28
11.3 NP-completeness	28
12. Exam Focus / Practice Checklist (MCQ-oriented)	29
12.1 DFA/NFA construction and minimization	29
12.2 NFA-to-DFA subset construction	29
12.3 RE $\leftrightarrow$ automaton conversion	29
12.4 Pumping-lemma non-regularity / non-CFL	29
12.5 Counting states in minimal DFA	29
12.6 Identifying language class	29
12.7 Closure-property true/false	29
12.8 Decidability/undecidability classification	29
12.9 Chomsky hierarchy mapping	29

How To Use These Notes

- These notes cover the **Theory of Computation (TOC)** portion of the **CIL MT (Coal India Limited - Management Trainee)** Systems-discipline professional-knowledge paper.
- The CIL MT exam is an **objective MCQ** computer-based test (CBT) of **conceptual-to-moderate** difficulty - generally easier than **GATE** but it samples the same syllabus.
- High-yield areas: **DFA/NFA construction**, **NFA-to-DFA subset construction**, **minimal DFA state counting**, **regular expression conversions**, **pumping lemma applicability**, **closure properties**, **language-class identification**, and **decidability classification**.
- Every red token is a likely **one-mark MCQ answer**: memorise the red items verbatim.
- The **Chomsky hierarchy** is the spine of the entire subject - learn the machine/grammar/language mapping first, then hang every other fact on it.

How the subject is structured

- **4** language classes in increasing power: **Regular** (subset of) **Context-Free** (subset of) **Context-Sensitive** (subset of) **Recursively Enumerable**.
- **4** machine models matched to them: **Finite Automata**, **Pushdown Automata**, **Linear Bounded Automata**, **Turing Machine**.
- **4** grammar types: **Type 3 (regular)**, **Type 2 (context-free)**, **Type 1 (context-sensitive)**, **Type 0 (unrestricted)**.

1. Foundations (Preliminaries)

1.1 Alphabets, Symbols and Strings

- An **alphabet** (denoted **Σ**) is a **finite, non-empty** set of **symbols**; example **$\Sigma = \{0,1\}$** (binary alphabet) or **$\Sigma = \{a,b\}$** .
- A **symbol** (also called a **letter**) is an atomic, indivisible element of the alphabet.
- A **string** (or **word**) is a **finite** ordered sequence of symbols drawn from an alphabet; example **0110** over **$\{0,1\}$** .
- The **length** of a string **w**, written **$|w|$** , is the **number of symbols** in it; **$|0110| = 4$** .
- **$|\epsilon| = 0$** - the empty string has length **zero**.

Core string operations

- **Concatenation**: joining string **x** and **y** to form **xy**; **$|xy| = |x| + |y|$** ; it is **associative** but **NOT commutative**.
- The **identity** element for concatenation is **epsilon** because **$w.\epsilon = \epsilon.w = w$** .
- **Reversal** of **w**, written **w^R** , reverses symbol order; **$(xy)^R = y^R x^R$** .
- A **prefix** of **w** is any leading portion; a **suffix** is any trailing portion; a **substring** (factor) is any contiguous portion.
- A **proper prefix/suffix/substring** is one that is **not equal** to the whole string and not **epsilon**.
- Number of **prefixes** of a string of length **n** is **$n+1$** (including **epsilon** and the whole string); same count for **suffixes**.
- Number of **substrings** (non-empty, by position) is **$n(n+1)/2$** ; including **epsilon** it is **$n(n+1)/2 + 1$** .
- Number of **distinct subsequences** can be up to **2^n** (subsequences need NOT be contiguous).

Powers of a string

- **$w^0 = \epsilon$, $w^1 = w$, $w^n = w^{(n-1)} w$** .
- **a^n** means the symbol **a** repeated **n** times.

Memory Trick

- "Prefix = front, Suffix = back, Substring = contiguous middle, Subsequence = skip allowed."

CBT Trap

- **Substring** must be **contiguous**; **subsequence** need **NOT** be contiguous - examiners swap these.
- **$|\epsilon| = 0$** but **$|\{\epsilon\}| = 1$** (a set containing the empty string has **one** element) - classic trick.
- Concatenation is **NOT commutative**: **$ab \neq ba$** in general.

1.2 The Empty String (Epsilon)

- **epsilon** (also written **lambda**) is the unique string of length **0**.
- **epsilon** is the **identity** for concatenation: **$\epsilon w = w \epsilon = w$** .
- The language **$\{\epsilon\}$** contains exactly **one** string and is **NOT** the empty language.
- The **empty language phi** (or **$\{\}$**) contains **zero** strings.

CBT Trap

- Distinguish **phi** (empty language, **0** strings) from **$\{\epsilon\}$** (language with **1** string) from **epsilon** (a string).
- **$\phi \cdot L = \phi$** (concatenation with empty language gives empty language), but **$\{\epsilon\} \cdot L = L$** .

1.3 Languages and Operations

- A **language L** over alphabet **Sigma** is any **subset** of **Sigma*** (i.e. **L subset of Sigma***); it may be **finite** or **infinite**.
- Union:** $L_1 \cup L_2 = \{ w : w \text{ in } L_1 \text{ OR } w \text{ in } L_2 \}$.
- Intersection:** $L_1 \cap L_2 = \{ w : w \text{ in } L_1 \text{ AND } w \text{ in } L_2 \}$.
- Concatenation:** $L_1.L_2 = \{ xy : x \text{ in } L_1, y \text{ in } L_2 \}$.
- Complement:** $L' = \text{Sigma}^* - L$ (all strings over the alphabet NOT in **L**).
- Difference:** $L_1 - L_2 = \{ w : w \text{ in } L_1 \text{ and } w \text{ not in } L_2 \}$.
- Reversal:** $L^R = \{ w^R : w \text{ in } L \}$.

Number facts

- The number of distinct languages over any non-empty alphabet is **uncountably infinite** ($2^{\aleph_0}$); the number of strings **Sigma*** is **countably infinite**.
- This counting gap is WHY some languages have **no** machine/grammar (there are more languages than programs).

CBT Trap

- Sigma*** is **countably infinite**; the set of all languages over **Sigma** is **uncountable**.
- $L \cdot \phi = \phi$ but $L \cup \phi = L$.

1.4 Kleene Star, Kleene Plus and Sigma-star

- Kleene star** $L^* = \bigcup_{i \geq 0} L^i = L^0 \cup L^1 \cup L^2 \cup \dots$; it ALWAYS includes **epsilon** (because $L^0 = \{\epsilon\}$).
- Kleene plus** $L^+ = \bigcup_{i \geq 1} L^i = L^1 \cup L^2 \cup \dots$; it includes **epsilon** ONLY if **epsilon** is already in **L**.
- Relationship: $L^+ = L.L^*$ and $L^* = L^+ \cup \{\epsilon\}$.
- Sigma*** = set of **ALL** finite strings over **Sigma**, including **epsilon**.
- Sigma+** = **Sigma*** - **{epsilon}** (all non-empty strings).
- $\phi^* = \{\epsilon\}$ - star of the empty language is the language containing only **epsilon**.
- $\{\epsilon\}^* = \{\epsilon\}$.

Operator	Includes epsilon?	Definition
L^*	Always	$\bigcup_{i \geq 0} L^i$
L^+	Only if epsilon in L	$\bigcup_{i \geq 1} L^i$
Sigma*	Always	all finite strings
Sigma+	Never	non-empty strings

Memory Trick

- "Star starts at zero (epsilon free); Plus starts at one (must earn epsilon)."

CBT Trap

- L^* always contains **epsilon** even if **L** does not - very common one-mark trap.
- $\phi^* = \{\epsilon\}$, NOT ϕ .

1.5 Chomsky Hierarchy

- Proposed by **Noam Chomsky (1956)**; classifies grammars into **4** types by production restrictions.

- **Type 0** = **Unrestricted grammar** -> **Recursively Enumerable** languages -> **Turing Machine**.
- **Type 1** = **Context-Sensitive grammar** -> **Context-Sensitive** languages -> **Linear Bounded Automaton (LBA)**.
- **Type 2** = **Context-Free grammar** -> **Context-Free** languages -> **Pushdown Automaton (PDA)**.
- **Type 3** = **Regular grammar** -> **Regular** languages -> **Finite Automaton (FA)**.
- Strict containment: **Regular subset CFL subset CSL subset REC subset RE**.

Type	Grammar	Production form	Machine	Language class				
Type 0	Unrestricted	alpha -> beta , alpha has a variable	TM	Recursively Enumerable				
Type 1	Context-sensitive	{{	alpha	<=	beta	}}	LBA	Context-Sensitive
Type 2	Context-free	A -> gamma (single variable LHS)	PDA	Context-Free				
Type 3	Regular	A -> aB or A -> Ba	Finite Automaton	Regular				

Production-form rules

- **Type 1**: every production **alpha -> beta** satisfies **|alpha| <= |beta|** (non-contracting); exception **S -> epsilon** allowed if **S** not on any RHS.
- **Type 2**: LHS is exactly **one** variable; RHS any string of terminals/variables.
- **Type 3**: RHS has at most **one** variable and it sits at the **same end** (all left-linear OR all right-linear).

Memory Trick

- "0 has no rules, 1 cannot shrink, 2 single variable, 3 one terminal one variable." Numbering: **lower type = more powerful**.

CBT Trap

- Numbering is **inverted**: **Type 0** is the **most** powerful, **Type 3** the **least**.
- **LBA** is the machine for **Type 1**; students wrongly say TM.
- A grammar that mixes left-linear and right-linear rules is **NOT** regular (it is **linear** / context-free).

2. Finite Automata

2.1 Deterministic Finite Automaton (DFA)

2.1.1 Formal definition (5-tuple)

- A DFA is a **5-tuple** $M = (Q, \Sigma, \delta, q_0, F)$.
- Q = finite set of **states**.
- Σ = finite **input alphabet**.
- δ = **transition function** $\delta: Q \times \Sigma \rightarrow Q$ (exactly **one** next state per (state, symbol)).
- q_0 = **start state**, $q_0 \in Q$.
- F = set of **accepting / final** states, $F \subseteq Q$.
- "Deterministic" means **exactly one** transition per symbol from each state - **no choice**, **no epsilon** moves.

2.1.2 Transition table and diagram

- The **transition diagram** is a directed graph: nodes = states, labelled edges = transitions, an arrow marks q_0 , double circles mark F .
- The **transition table** lists δ as a matrix indexed by state (rows) and symbol (columns).
- A DFA with n states and k input symbols has a table of $n \times k$ entries and must define **all** of them (totality).

2.1.3 String acceptance; language of a DFA

- Extended transition $\delta^*(q, \epsilon) = q$ and $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$.
- String w is **accepted** iff $\delta^*(q_0, w) \in F$.
- Language of M**: $L(M) = \{ w \in \Sigma^* : \delta^*(q_0, w) \in F \}$.
- A language is **regular** iff it is $L(M)$ for some DFA.

2.1.4 DFA construction for given languages

- Standard pattern - "strings whose length mod $n = r$ ": use n states arranged in a cycle.
- "Number of a's is even / divisible by k ": k states counting modulo k .
- "Contains substring w ": states track **longest prefix of w** matched so far ($|w|+1$ states).
- "Ends with w ": similar prefix-tracking automaton.
- "Binary number divisible by k ": k states tracking **remainder** mod k ; on reading bit b , new remainder = $(2 \cdot \text{old} + b) \bmod k$.

2.1.5 Dead / trap states

- A **dead state** (trap / sink) is a **non-accepting** state whose **every** transition loops back to itself - once entered, acceptance is **impossible**.
- Dead states are needed to make a **partial** DFA **total** (complete).
- A **minimal** DFA has at most **one** dead state.

Feature	DFA
Transitions per symbol	exactly 1
Epsilon moves	not allowed
Backtracking	none
Acceptance test	single path ends in F

Memory Trick

- "DFA = Definite, one Door per symbol from every room."

CBT Trap

- A DFA must be **complete**: missing transitions imply an implicit **dead state** - count it when counting states.
- **delta** co-domain is a **single** state **Q**, NOT 2^Q (that is the NFA difference).
- Empty string **epsilon** is accepted iff **q0 in F**.

2.2 Non-deterministic Finite Automaton (NFA)

2.2.1 Formal definition; multiple transitions

- An NFA is a **5-tuple** $M = (Q, \Sigma, \delta, q_0, F)$ where **delta**: $Q \times \Sigma \rightarrow 2^Q$ (the **power set**).
- From a state on one symbol the NFA may go to **zero, one, or many** states.

2.2.2 Acceptance (existence of accepting path)

- An NFA **accepts w** iff **at least one** computation path ends in a state of **F**.
- Equivalently $\delta^+(q_0, w) \cap F \neq \emptyset$ where δ^+ returns a **set** of states.
- Rejection requires **ALL** paths to fail.

2.2.3 NFA construction

- NFAs are easier to design for "contains/ends-with" patterns because you can **guess** where the pattern starts.
- Every **DFA** is trivially an **NFA**; the converse needs subset construction.
- **Theorem**: DFA and NFA recognise **exactly the same** class - the **regular languages** (equivalent in power).

Aspect	DFA	NFA
delta range	Q	2^Q (power set)
Moves per symbol	1	0, 1, or many
Epsilon moves	No	only in epsilon-NFA
Power	Regular	Regular (same)
Size	may be larger	usually smaller
Execution	one path	all paths / guess

Memory Trick

- "NFA guesses and is forgiven if ANY guess works."

CBT Trap

- NFA and DFA are **equally powerful** - non-determinism adds **convenience**, not **power**, for finite automata.
- An NFA need **not** define a transition for every symbol (missing = go to **empty set** = that path dies).

2.3 Epsilon-NFA

- An **epsilon-NFA** allows transitions on **epsilon** (no input consumed): **delta**: $Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$.
- **epsilon-closure(q)** = set of states reachable from **q** using **epsilon** moves only (including **q** itself).
- Acceptance: **w** accepted iff **epsilon-closure** of the reachable set intersects **F**.
- **epsilon-NFA**, **NFA**, and **DFA** are all **equivalent** (recognise the regular languages).
- Epsilon moves make **union** and **concatenation** constructions (Thompson) trivial.

CBT Trap

- **epsilon-closure(q)** always includes **q** itself - do not forget it.
- Epsilon transitions add **no** extra language power, only structural convenience.

2.4 Conversions

2.4.1 NFA to DFA (Subset Construction)

- Algorithm: DFA states are **subsets** of NFA states; start = **{q0}**; for each subset and symbol, new state = **union of delta over members**.
- A DFA subset is **accepting** iff it contains **at least one** NFA final state.
- Worst-case DFA size from an **n**-state NFA is **2ⁿ** states (**exponential blow-up**).
- Also called the **powerset / subset construction**.

2.4.2 Epsilon-NFA to DFA

- Same subset construction but each subset is **epsilon-closed**: start = **epsilon-closure(q0)**; transition = **epsilon-closure(union of delta)**.

2.4.3 State blow-up (concept)

- For some languages the **minimal DFA** truly needs **2ⁿ** states although an NFA needs only **n** (e.g. "the **n**-th symbol from the end is **1**" needs **2ⁿ** DFA states but **n+1** NFA states).

Conversion	Method	Worst-case size
NFA -> DFA	subset construction	2ⁿ
epsilon-NFA -> DFA	epsilon-closure + subset	2ⁿ
DFA -> NFA	trivial (DFA is an NFA)	n

CBT Trap

- The **n**-th-from-end language: NFA = **n+1** states, minimal DFA = **2ⁿ** states - frequent numeric MCQ.
- Subset construction can produce **unreachable** subsets; count only **reachable** ones.

2.5 DFA Minimization

- Goal: find the **unique** (up to renaming) DFA with the **fewest** states for a regular language (**Myhill-Nerode** guarantees uniqueness).
- **Table-filling (Myhill-Nerode) algorithm**: mark pairs where one is **final** and the other **non-final**; iteratively mark pair **(p,q)** if **(delta(p,a), delta(q,a))** is already marked for some **a**; unmarked pairs are **equivalent** and merged.
- Two states are **distinguishable** if some string leads one to **accept** and the other to **reject**; otherwise **equivalent / indistinguishable**.
- **Hopcroft's algorithm** minimises in **O(n log n)** time (best known); table-filling is **O(n²)** per pass.
- Remove **unreachable** states **before** minimising.

Steps (table-filling)

1. Remove unreachable states.
2. Mark all **(final, non-final)** pairs.
3. Repeat: mark **(p,q)** if any symbol sends them to an already-marked pair.
4. Merge all **unmarked** pairs into single states.

Memory Trick

- "Distinguish = some future string tells them apart."

CBT Trap

- Always remove **unreachable** states first - they are not part of the minimal DFA but inflate counts.
- The minimal DFA is **unique** up to renaming (Myhill-Nerode) - a favourite true/false item.
- Number of **distinct equivalence classes** of the Myhill-Nerode relation = number of states in the **minimal DFA**.

2.6 Moore and Mealy Machines

- These are **finite-state transducers** (they produce **output**), not just acceptors.
- **Moore machine**: output depends **only on the current state**; output function **lambda: $Q \rightarrow O$** ; defined as a **6-tuple $(Q, \Sigma, O, \delta, \lambda, q_0)$** .
- **Mealy machine**: output depends on **current state AND input**; output function **lambda: $Q \times \Sigma \rightarrow O$** ; also a **6-tuple**.
- For an input of length **n**: a **Moore** machine emits **n+1** outputs (one for the start state too); a **Mealy** machine emits **n** outputs.
- Moore and Mealy are **equivalent** in expressive power; a Moore with **m** states converts to a Mealy with **m** states, and a Mealy converts to a Moore with at most **$m * |O|$** states.

Feature	Moore	Mealy
Output depends on	state only	state + input
Output count for length n	n+1	n
Output placed on	states	transitions
Reaction speed	slower (one step delay)	faster
States needed	usually more	usually fewer

Memory Trick

- "Moore = Output On states (more outputs n+1); Mealy = Output on Moves (n outputs)."

CBT Trap

- **Moore** outputs **n+1** symbols, **Mealy** outputs **n** - exact count is a classic MCQ.
- Transducers do **not** have final states **F**; they map inputs to outputs.

3. Regular Expressions

3.1 Definition; operators and precedence

- A **regular expression (RE)** is an algebraic notation for a regular language, built from **phi**, **epsilon**, and symbols **a** in **Sigma**.
- Base cases: **phi** denotes $\{\}$, **epsilon** denotes $\{\epsilon\}$, **a** denotes $\{a\}$.
- Inductive operators: **union (+ or |)**, **concatenation (.)**, **Kleene star (*)**.
- **Operator precedence (highest to lowest):** **star (*)** then **concatenation** then **union (+)**.
- Parentheses override precedence.

Mapping to languages

- $L(r+s) = L(r) \cup L(s)$, $L(rs) = L(r)L(s)$, $L(r^*) = (L(r))^*$.

3.2 Algebraic identities

- $r + s = s + r$ (union commutative).
- $(r+s)+t = r+(s+t)$ and $(rs)t = r(st)$ (associativity).
- $r + r = r$ (idempotence of union).
- $\epsilon r = r \epsilon = r$ (epsilon identity for concatenation).
- $\phi r = r \phi = \phi$ (phi annihilates concatenation).
- $\phi + r = r$ (phi identity for union).
- $r(s+t) = rs + rt$ (left distributivity); $(s+t)r = sr + tr$.
- $(r^*)^* = r^*$, $\phi^* = \epsilon$, $\epsilon^* = \epsilon$.
- $r^* = \epsilon + r r^* = \epsilon + r^* r$.
- $(r + \epsilon)^* = r^*$ and $r^* r^* = r^*$.
- $(rs)^* r = r (sr)^*$ (shifting identity).

CBT Trap

- $(r+s)^* \neq r^* + s^*$ in general - a very common false-identity trap.
- $(rs)^* \neq r^* s^*$ in general.
- Concatenation does **not** commute: $rs \neq sr$.

3.3 RE to Finite Automaton

- **Thompson's construction** builds an **epsilon-NFA** recursively from the RE; each operator adds a small gadget with **epsilon** edges.
- An RE of size **n** yields an epsilon-NFA with **O(n)** states.
- Then convert **epsilon-NFA** $\rightarrow$ **DFA** if a DFA is required.

3.4 Finite Automaton to RE

- **Arden's theorem**: the equation $R = Q + RP$ has the unique solution $R = QP^*$ provided **P** does **not** contain **epsilon** (i.e. **epsilon not in L(P)**).
- **State-elimination method**: repeatedly remove intermediate states, relabelling edges with REs, until a single RE from start to final remains.
- **Kleene's theorem**: a language is **regular** iff it is described by a **regular expression** iff it is accepted by a **finite automaton**.

Memory Trick

- "Arden: $R = Q + RP$ gives $R = Q P^*$ (P must be epsilon-free for uniqueness)."

CBT Trap

- Arden's uniqueness needs **epsilon not in P**; if **epsilon in P** the solution is **not unique**.
- **Kleene's theorem** ties together **RE = FA = regular language** - all three are equivalent.

4. Properties of Regular Languages

4.1 Closure Properties

- Regular languages are closed under: **union**, **intersection**, **complement**, **concatenation**, **Kleene star**, **reversal**, **difference**, **homomorphism**, **inverse homomorphism**, and **prefix/suffix/quotient**.
- Closure under **complement**: swap **final** and **non-final** states of a **complete DFA**.
- Closure under **intersection**: **product (cross) construction** of two DFAs with **m** and **n** states gives at most **m*n** states.
- Regular languages form a **Boolean algebra** (closed under all Boolean operations).

Operation	Regular closed?
Union	Yes
Intersection	Yes
Complement	Yes
Concatenation	Yes
Kleene star	Yes
Reversal	Yes
Difference	Yes
Homomorphism	Yes

CBT Trap

- Regular languages are closed under **ALL** the common operations - if an MCQ offers "not closed under intersection" for regular, it is **false** (that is the **CFL** property).

4.2 Pumping Lemma for Regular Languages

- Statement**: for every regular language **L** there is a **pumping length** $p \geq 1$ such that any **w** in **L** with $|w| \geq p$ can be split as **w = xyz** with (1) $|y| \geq 1$, (2) $|xy| \leq p$, and (3) $x y^i z$ in **L** for all $i \geq 0$.
- It is a **necessary** (not sufficient) condition - used to **prove non-regularity** by **contradiction**.
- Classic non-regular language proven by it: **L = { aⁿ bⁿ : n ≥ 0 }** (requires unbounded **counting**, impossible with finite memory).
- Also non-regular: **{ aⁿ bⁿ cⁿ }, { ww }, { a^p : p prime }, { a^(n²) }, balanced parentheses.**

Memory Trick

- "Pumping = repeat the middle y; if it ever leaves L, L is not regular." Conditions: $|y| \geq 1$, $|xy| \leq p$, $xy^i z$ in **L**.

CBT Trap

- The pumping lemma can **only prove non-regularity**; passing it does **NOT** prove regularity (not sufficient).
- $|xy| \leq p$ forces **y** to lie within the **first p** symbols - the key to the proof.

4.3 Myhill-Nerode Theorem

- Statement**: **L** is regular iff the **Myhill-Nerode equivalence relation** ($x \sim y$ iff for all **z**, xz in **L** $\iff$ yz in **L**) has a **finite** number of equivalence classes.
- The number of classes equals the number of states in the **minimal DFA** (the **index** of the relation).
- It is both a **necessary AND sufficient** test for regularity (unlike the pumping lemma).

- It also proves the **minimal DFA is unique**.

CBT Trap

- Myhill-Nerode is **necessary and sufficient**; the pumping lemma is **only necessary** - the key distinction.
- Infinite index => **not regular**.

4.4 Decision Properties

- **Membership** ("is w in L ?"): **decidable**; simulate the DFA in $O(|w|)$ time.
- **Emptiness** ("is $L = \phi$?"): **decidable**; check if any **final** state is **reachable** from q_0 (graph reachability).
- **Finiteness** ("is L finite?"): **decidable**; L is infinite iff the DFA has a **cycle** on a path from start to a reachable final state.
- **Equivalence** ("is $L_1 = L_2$?"): **decidable**; minimise both DFAs and compare, or test $(L_1 - L_2) \cup (L_2 - L_1) = \phi$.

Problem	Regular	Decidable?
Membership	yes	Yes
Emptiness	yes	Yes
Finiteness	yes	Yes
Equivalence	yes	Yes

CBT Trap

- **ALL** standard decision problems for regular languages are **decidable** - including **equivalence** (which is **undecidable** for CFLs).

4.5 Regular Grammars

- A **right-linear grammar** has all productions of the form $A \rightarrow aB$ or $A \rightarrow a$ or $A \rightarrow \epsilon$.
- A **left-linear grammar** has all productions of the form $A \rightarrow Ba$ or $A \rightarrow a$.
- Both right-linear and left-linear grammars generate exactly the **regular languages** and are equivalent to **finite automata**.
- **Mixing** left- and right-linear rules in one grammar can generate **non-regular** (context-free) languages.

CBT Trap

- A **linear grammar** (at most one variable on the RHS, but possibly in the **middle**) is **more** general than regular - it is **context-free**, e.g. $S \rightarrow aSb$.
- Pure **left-linear** = pure **right-linear** = **regular**; mixed = **not necessarily regular**.

5. Context-Free Grammars (CFG)

5.1 Formal definition

- A CFG is a **4-tuple** $G = (V, T, P, S)$.
- V = finite set of **variables** (non-terminals).
- T = finite set of **terminals**, with $V \cap T = \emptyset$.
- P = finite set of **productions** of the form $A \rightarrow \alpha$ where $A \in V$ and $\alpha \in (V \cup T)^*$.
- $S \in V$ = **start symbol**.
- The **language** $L(G) = \{ w \in T^* : S \Rightarrow^* w \}$ (strings derivable from S).

5.2 Derivations: leftmost and rightmost

- A **leftmost derivation (LMD)** always rewrites the **leftmost** variable first.
- A **rightmost derivation (RMD)** (canonical) always rewrites the **rightmost** variable first.
- $\Rightarrow$ denotes one step, $\Rightarrow^*$ denotes zero-or-more steps.
- Each **parse tree** corresponds to **exactly one** LMD and **exactly one** RMD.

5.3 Parse trees and yield

- A **parse tree (derivation tree)**: root = S , internal nodes = variables, leaves = terminals or **epsilon**.
- The **yield** is the string of leaves read **left to right**.
- Distinct parse trees for the same string $\Rightarrow$ grammar is **ambiguous**.

5.4 Ambiguity

- A grammar is **ambiguous** if some string has ≥ 2 distinct **parse trees** (equivalently ≥ 2 leftmost derivations).
- Ambiguity is a property of the **grammar**, not the language.
- A CFL is **inherently ambiguous** if **every** CFG for it is ambiguous; example $\{ a^i b^j c^k : i=j \text{ OR } j=k \}$.
- Checking whether an **arbitrary** CFG is ambiguous is **undecidable**.

CBT Trap

- Ambiguity is undecidable in general; and **inherent ambiguity** is a property of the **language** (no unambiguous grammar exists).
- A grammar can be **ambiguous** even if the language has an **unambiguous** grammar - rewrite to fix.

5.5 CFG design for given languages

- $\{ a^n b^n \}$: $S \rightarrow aSb \mid \epsilon$.
- $\{ a^n b^m : n=m \}$ same as above; $\{ a^n b^m : n \geq m \}$: $S \rightarrow aSb \mid aS \mid \epsilon$.
- Palindromes over $\{a,b\}$: $S \rightarrow aSa \mid bSb \mid a \mid b \mid \epsilon$.
- Balanced parentheses: $S \rightarrow SS \mid (S) \mid \epsilon$.
- Equal a's and b's: $S \rightarrow aSb \mid bSa \mid SS \mid \epsilon$.

5.6 Simplification of CFGs

- **Order matters**: remove **epsilon-productions**, then **unit productions**, then **useless symbols** (a common recommended order to avoid reintroducing removed forms).

- **Useless symbols** = those that are **non-generating** (derive no terminal string) OR **unreachable** from **S**; remove **non-generating** first, then **unreachable**.
- **Epsilon-production**: $A \rightarrow \epsilon$; eliminate by adding versions of productions with nullable variables omitted (keep $S \rightarrow \epsilon$ only if **epsilon in L**).
- **Unit production**: $A \rightarrow B$ (single variable); eliminate via **unit pairs**.

CBT Trap

- A symbol is **useless** if it is **non-generating** OR **unreachable** - both conditions matter.
- Remove **non-generating** symbols **before unreachable** ones, else you may keep junk.

5.7 Normal Forms

- **Chomsky Normal Form (CNF)**: every production is $A \rightarrow BC$ (two variables) or $A \rightarrow a$ (single terminal); $S \rightarrow \epsilon$ allowed only if **epsilon in L**.
- A CNF derivation of a string of length $n \geq 1$ uses exactly $2n - 1$ production steps; a parse tree has at most depth-related bounds used by **CYK**.
- **Greibach Normal Form (GNF)** (advanced - awareness): every production is $A \rightarrow a \alpha$ where **alpha** is a (possibly empty) string of **variables**; a string of length n derives in exactly n steps.

CBT Trap

- In **CNF** a string of length n needs exactly $2n-1$ steps - a frequent numeric MCQ.
- **GNF** begins each RHS with **exactly one terminal**; it is used to convert CFG to PDA with no epsilon stack-pop ambiguity.

5.8 Left recursion and left factoring

- **Left recursion**: $A \rightarrow A \alpha$ (immediate) breaks **top-down** (recursive-descent / **LL**) parsers; eliminated by rewriting to **right recursion**: $A \rightarrow \beta A'$, $A' \rightarrow \alpha A' \mid \epsilon$.
- **Left factoring**: factor common prefixes $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$ into $A \rightarrow \alpha A'$, $A' \rightarrow \beta_1 \mid \beta_2$ to enable **predictive** parsing.

CBT Trap

- **Left recursion** and **ambiguity** are different problems - eliminating left recursion does **not** remove ambiguity.

6. Pushdown Automata (PDA)

6.1 Formal definition (7-tuple); stack operations

- A PDA is a **7-tuple** $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$.
- Q = states, Σ = input alphabet, Γ = **stack alphabet**.
- $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow 2^*(Q \times \Gamma^*)$ (finite subsets).
- q_0 = start state, Z_0 = **initial stack symbol**, F = final states.
- A PDA = **finite automaton + a stack (LIFO, unbounded)** - the stack gives the extra **memory** that recognises CFLs.
- An **instantaneous description (ID)** is (state, remaining input, stack contents).

6.2 Acceptance: final state vs empty stack

- Acceptance by final state**: reach a state in F after consuming input.
- Acceptance by empty stack**: stack becomes **empty** after consuming input (F is irrelevant / taken as ϕ).
- The two modes are **equivalent** for **NPDA**: each recognises **exactly** the CFLs.

6.3 NPDA construction from CFG

- Standard construction yields a **single-state** PDA accepting by **empty stack**: push the start variable, then repeatedly replace a variable on top by a production RHS, and match terminals against input.

6.4 Deterministic PDA and DCFL

- A **DPDA** has at most **one** move in each configuration and no conflicting **epsilon** move; it recognises **Deterministic Context-Free Languages (DCFL)**.
- DCFL** is a **proper subset** of **CFL**.
- DCFLs (accepted by final state) are closed under **complement** but **not** under **union** or **intersection**.

6.5 Equivalence of CFG and PDA

- Theorem**: a language is **context-free** iff it is accepted by some **NPDA**; CFG and NPDA are **equivalent**.

6.6 NPDA more powerful than DPDA

- NPDA** is strictly **more powerful** than **DPDA**: e.g. the language of **even-length palindromes** $\{ w w^R \}$ is a CFL accepted by an NPDA but **NOT** by any DPDA.
- This contrasts with finite automata where **NFA = DFA** in power.

Feature	DFA/NFA	DPDA	NPDA
Memory	none	1 stack	1 stack
Class	Regular	DCFL	CFL
Determinism adds power?	No (NFA=DFA)	-	Yes (NPDA>DPDA)
Closed under complement	Yes	Yes	No

Memory Trick

- "One stack = one counting job; that is why $a^n b^n$ is a CFL but $a^n b^n c^n$ is not (needs two counts)."

CBT Trap

- For PDAs **non-determinism ADDS power** (**NPDA > DPDA**); for finite automata it does **not** (**NFA = DFA**).
- A **single stack** cannot compare **three** quantities - **$a^n b^n c^n$** is **not** context-free.
- **Final-state** and **empty-stack** acceptance are equivalent only for **NPDA**, not necessarily for DPDA.

7. Properties of Context-Free Languages

7.1 Closure properties

- CFLs are closed under: **union**, **concatenation**, **Kleene star**, **reversal**, and **homomorphism**.
- CFLs are **NOT** closed under: **intersection**, **complement**, or **difference**.
- Counterexample for intersection: $\{a^n b^n c^m\} \cap \{a^m b^n c^n\} = \{a^n b^n c^n\}$ which is **not** a CFL.

Operation	CFL closed?
Union	Yes
Concatenation	Yes
Kleene star	Yes
Reversal	Yes
Intersection	No
Complement	No
Intersection with Regular	Yes

7.2 CFL intersect Regular = CFL

- The intersection of a **CFL** with a **regular** language is always a **CFL** (run the PDA and DFA in parallel - the **product** keeps one stack).

7.3 Pumping Lemma for CFLs

- Statement:** for every CFL there is p such that any $z \in L$ with $|z| \geq p$ can be written $z = uvwxy$ with (1) $|vwx| \leq p$, (2) $|vx| \geq 1$, and (3) $u v^i w x^i y \in L$ for all $i \geq 0$.
- It pumps **two** substrings v and x **simultaneously** (the regular version pumps **one**).
- Used to prove $\{a^n b^n c^n\}$, $\{ww\}$, $\{a^{(n^2)}\}$, $\{a^p : p \text{ prime}\}$ are **not** context-free.

CBT Trap

- CFL pumping pumps **two** pieces (v and x); regular pumping pumps **one** (y).
- $\{a^n b^n c^n\}$ is **not** a CFL but **IS** context-sensitive.

7.4 Decision properties of CFLs

- Membership:** **decidable** - **CYK algorithm** runs in $O(n^3)$ time using a **CNF** grammar ($n = |w|$).
- Emptiness:** **decidable** - check if S is a **generating** symbol.
- Finiteness:** **decidable** - look for **cycles** in the dependency graph of a cleaned CNF grammar.
- Equivalence** of two CFGs: **UNDECIDABLE**.
- $L_1 \cap L_2 = \emptyset$** (CFL disjointness): **UNDECIDABLE**.
- Is $L = \Sigma^*$ / is L regular / is L ambiguous / is L a CFL:** **UNDECIDABLE**.

Problem	CFL	Decidable?
Membership (CYK)	yes	Yes, $O(n^3)$
Emptiness	yes	Yes
Finiteness	yes	Yes
Equivalence	yes	No (undecidable)

Problem	CFL	Decidable?
L = Sigma*	yes	No
Ambiguity	yes	No

Memory Trick

- "For CFLs: membership/emptiness/finiteness DECIDABLE; equivalence/totality/ambiguity UNDECIDABLE."

CBT Trap

- **Equivalence** is **decidable** for regular languages but **undecidable** for CFLs - the single most-tested contrast.
- **CYK** complexity is **$O(n^3)$** - memorise the cube.

8. Turing Machines (TM)

8.1 Formal definition (7-tuple)

- A TM is a **7-tuple** $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$.
- Q = states, Σ = input alphabet, Γ = **tape alphabet** with Σ subset of Γ .
- B in Γ = **blank symbol**, B not in Σ .
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ (for a standard deterministic TM): write a symbol and move head **Left** or **Right**.
- q_0 = start state, F = accepting states.
- The tape is **infinite** (at least one direction) and read/write; the head moves one cell per step.

8.2 TM as language acceptor; configurations

- A **configuration** / **ID** captures **tape contents**, **head position**, and **current state**.
- A TM **accepts** w if it enters an **accept** state; it may **reject** (halt in reject) or **loop forever**.
- $L(M)$ = set of strings the TM **accepts**.

8.3 TM as computing device

- TMs compute **functions**: input on the tape, output left on the tape at halt - the formal model of **algorithm** / **effective computation**.

8.4 Construction of simple TMs

- Build TMs for tasks like $\{a^n b^n c^n\}$ (cross off one of each per pass), **copying**, **incrementing a binary number**, **comparing** two blocks - by marking symbols and sweeping.

8.5 Variants and equivalence

- **Multi-tape TM**, **Non-deterministic TM (NTM)**, **multi-track TM**, **two-way / one-way infinite tape**, and **enumerator** are all **equivalent** in power to the standard single-tape TM (they accept exactly the **RE** languages).
- A k -tape TM running in time t is simulated by a single-tape TM in $O(t^2)$ time.
- An **NTM** running in time t is simulated by a deterministic TM in $O(2^t)$ (exponential) time but recognises the **same** class of languages.
- An **enumerator** lists a language's strings; a language is **RE** iff some enumerator enumerates it; it is **recursive** iff some enumerator lists it in **lexicographic (canonical)** order.

CBT Trap

- All TM variants are **equally powerful** (recognise **RE**); non-determinism adds **no language power** to TMs (only possible **time** speed-up).

8.6 Universal Turing Machine

- A **Universal Turing Machine (UTM)** takes an encoding $\langle M, w \rangle$ of any TM M and input w , and **simulates** M on w .
- The UTM is the theoretical basis of the **stored-program computer**.
- The language $L_u = \{ \langle M, w \rangle : M \text{ accepts } w \}$ is **RE but not recursive** (the **universal language**).

8.7 Church-Turing Thesis

- **Church-Turing thesis**: any function that is **effectively / algorithmically** computable can be computed by a **Turing Machine**.
- It is a **thesis (not a theorem)** - cannot be proved, only supported by the equivalence of **lambda-calculus**, **recursive functions**, and **TMs**.

CBT Trap

- The Church-Turing thesis is a **hypothesis**, not a provable theorem.
- **L_u** (universal language) is **RE but not recursive** - a key example.

9. Recursive and Recursively Enumerable Languages

9.1 Recursive (decidable) languages

- A language is **recursive (decidable)** if some TM **halts on every input** and decides membership (always says **yes** or **no**) - a **total** TM / **decider**.
- Recursive languages are also called **Turing-decidable**.

9.2 Recursively enumerable (semi-decidable) languages

- A language is **recursively enumerable (RE)** / **semi-decidable** / **Turing-recognisable** if some TM **accepts** every string in **L** but may **loop forever** on strings **not** in **L**.
- Every **recursive** language is **RE**; the converse is **false** (**REC subset of RE**, proper).

9.3 Closure properties

- Recursive** languages are closed under: **union**, **intersection**, **complement**, **concatenation**, **star**.
- RE** languages are closed under: **union**, **intersection**, **concatenation**, **star** - but **NOT** under **complement**.

Operation	Recursive	RE
Union	Yes	Yes
Intersection	Yes	Yes
Complement	Yes	No
Concatenation	Yes	Yes
Kleene star	Yes	Yes

9.4 Recursive iff L and complement both RE

- Theorem (Post)**: **L** is **recursive** iff **both L** and its **complement L'** are **RE**.
- Consequence: if **L** is **RE** but **not** recursive, then **L'** is **NOT RE** (e.g. **L_u** is RE, its complement is not RE).

CBT Trap

- RE** is **NOT** closed under **complement**; **Recursive IS** closed under complement - the deciding distinction.
- If **L** and **L'** are **both RE** then **L** is **recursive**.

10. Decidability and Undecidability

10.1 Decidable problems for FA and CFG

- **Decidable for DFA/regular**: membership, emptiness, finiteness, **equivalence**, universality.
- **Decidable for CFG/CFL**: membership (**CYK $O(n^3)$**), emptiness, finiteness.
- **Undecidable for CFG**: **equivalence**, **ambiguity**, **$L = \Sigma^*$** , **is L regular**, intersection-emptiness.

Problem	DFA	CFG	TM
Membership	Dec	Dec	Undec (RE)
Emptiness	Dec	Dec	Undec
Equivalence	Dec	Undec	Undec
Finiteness	Dec	Dec	Undec

10.2 Halting problem

- The **Halting Problem** ("does TM **M** halt on input **w**?") is **UNDECIDABLE** - proven by **Alan Turing (1936)** via **diagonalization**.
- **$HALT_TM = \{ \langle M, w \rangle : M \text{ halts on } w \}$** is **RE but not recursive**.

10.3 Reductions (mapping reducibility)

- A **mapping (many-one) reduction $A \leq_m B$** is a **computable** function **f** with **x in A** iff **f(x) in B**.
- If **$A \leq_m B$** and **B** is **decidable**, then **A** is **decidable**; contrapositive: if **A** is **undecidable**, so is **B**.
- Reductions are the standard tool to **prove** new problems undecidable (reduce the Halting problem to them).

10.4 Rice's Theorem

- **Rice's theorem**: every **non-trivial** property of the **language** recognised by a TM is **UNDECIDABLE**.
- "Non-trivial" = the property holds for **some** but **not all** RE languages.
- Examples (all **undecidable**): "is **L(M)** empty?", "is **L(M)** regular?", "is **L(M)** finite?", "does **L(M)** contain a given string?".

CBT Trap

- Rice's theorem is about properties of the **language** (semantic), not the **machine** (syntactic, e.g. "does M have 5 states?" is decidable).
- **Trivial** properties (always true or always false) **are** decidable.

10.5 Post Correspondence Problem (PCP)

- **Post Correspondence Problem (PCP)**: given pairs of strings, is there a **sequence of indices** making the top concatenation equal the bottom? - **UNDECIDABLE**.
- Used to prove many CFG problems (ambiguity, intersection-emptiness) undecidable.

10.6 RE-but-not-recursive examples

- **L_u** (universal language), **$HALT_TM$** (halting problem), **$A_TM = \{ \langle M, w \rangle : M \text{ accepts } w \}$** are all **RE but not recursive**.
- The **complement** of any of these is **not even RE**.

- The **diagonal language L_d** (machines that do **not** accept their own encoding) is **not RE**.

Language	Class
A_{TM} / L_u	RE, not recursive
$HALT_{TM}$	RE, not recursive
Complement of A_{TM}	not RE
L_d (diagonal)	not RE
$\{a^n b^n c^n\}$	recursive (CSL)

Memory Trick

- "Halting, Universal, Acceptance = RE but not recursive; their complements = not RE."

CBT Trap

- **A_{TM}** is **RE**; its **complement** is **not RE** - if both were RE, **A_{TM}** would be recursive (it is not).

11. Introduction to Complexity (Awareness)

11.1 Time and space complexity of TMs

- **Time complexity** = number of **steps** the TM takes as a function of input length **n**; **space** = number of **tape cells** used.
- **DTIME(t)** / **NTIME(t)** and **DSPACE(s)** / **NSPACE(s)** are the basic resource-bounded classes.

11.2 Classes P and NP

- **P** = problems decidable by a **deterministic** TM in **polynomial** time $O(n^k)$.
- **NP** = problems decidable by a **non-deterministic** TM in **polynomial** time = problems whose **yes-certificate** is **verifiable** in polynomial time.
- **P subset of NP**; whether **P = NP** is the famous **open** problem (a **Millennium Prize** question).

11.3 NP-completeness

- **NP-complete** = problems that are in **NP** AND every NP problem reduces to them in **polynomial** time (**NP-hard** + in NP).
- First proven NP-complete problem: **SAT** (**Cook-Levin theorem, 1971**).
- Classic NP-complete problems: **3-SAT**, **Clique**, **Vertex Cover**, **Hamiltonian Cycle**, **TSP (decision)**, **Subset-Sum**, **Graph Colouring**.
- **NP-hard** = at least as hard as every NP problem, but **need not be in NP** (e.g. the **Halting problem** is NP-hard but undecidable).

Class	Definition
P	poly-time deterministic
NP	poly-time verifiable / non-deterministic
NP-complete	in NP and NP-hard
NP-hard	every NP problem reduces to it

CBT Trap

- **NP** stands for **Non-deterministic Polynomial**, NOT "non-polynomial".
- **NP-hard** problems need **not** be in NP (or even decidable); **NP-complete** = NP-hard **AND** in NP.
- **Cook-Levin** established **SAT** as the first NP-complete problem.

12. Exam Focus / Practice Checklist (MCQ-oriented)

12.1 DFA/NFA construction and minimization

- Minimal DFA for "**divisible by k** " = k states; for "ends with a fixed string w " = $|w|+1$ states (if no overlaps reduce it).
- Always make the DFA **complete** (add a **dead state**) before counting.

12.2 NFA-to-DFA subset construction

- DFA states = **reachable subsets**; an n -state NFA gives at most 2^n DFA states.

12.3 RE $\leftrightarrow$ automaton conversion

- RE $\rightarrow$ NFA: Thompson; NFA $\rightarrow$ DFA: subset; DFA $\rightarrow$ RE: state elimination / Arden.**

12.4 Pumping-lemma non-regularity / non-CFL

- $a^n b^n$ non-regular but CFL; $a^n b^n c^n$ and ww non-CFL; a^p (p prime) and $a^{(n^2)}$ non-regular and non-CFL.

12.5 Counting states in minimal DFA

- " n -th symbol from the end" needs 2^n DFA states; "divisible by k " needs k states.

12.6 Identifying language class

- Single counting / nesting $\Rightarrow$ **CFL**; two independent counts ($a^n b^n c^n$) $\Rightarrow$ **CSL**; finite memory pattern $\Rightarrow$ **Regular**.

12.7 Closure-property true/false

- Regular**: closed under everything common. **CFL**: NOT under intersection/complement. **Recursive**: closed under complement. **RE**: NOT under complement.

12.8 Decidability/undecidability classification

- Regular & CFL standard problems mostly **decidable** (except CFL equivalence/ambiguity); TM problems mostly **undecidable (Rice)**.

12.9 Chomsky hierarchy mapping

- Type 3 = FA = Regular; Type 2 = PDA = CFL; Type 1 = LBA = CSL; Type 0 = TM = RE.**

Final rapid-fire memory table

Class	Machine	Grammar	Closed under complement?	Equivalence decidable?
Regular	FA	Type 3	Yes	Yes
CFL	PDA	Type 2	No	No
CSL	LBA	Type 1	Yes	No

Class	Machine	Grammar	Closed under complement?	Equivalence decidable?
Recursive	TM (halting)	-	Yes	No
RE	TM	Type 0	No	No

CBT Trap

- The most-tested single fact: **equivalence** is **decidable for regular** but **undecidable for CFL**.
- The second: **NFA = DFA** in power but **NPDA > DPDA** in power.
- The third: **$a^n b^n$** is **CFL not regular**; **$a^n b^n c^n$** is **CSL not CFL**.